

The @hss Class: Audit and Plan for New Overloads

what works, what is broken, and which MATLAB functions to add next

Arjun Sethi-Olowin (audit prepared with Claude)

October 2026

1 Summary

I read every file in @hss/ and +hssutil/, ran the existing tests, and probed the class with edge cases the tests do not cover. Everything was run on the repository code as it is (no file was modified), in GNU Octave 8.4 through octave_compat; items that depend on MATLAB itself are listed in Section 8 for you to confirm in R2025b.

- **The existing tests pass.** All 109 test_hss methods (2160 checks; run in Octave through a small stand-in for matlab.unittest) pass, except two checks that expect MATLAB’s validator error identifier and one test that needs inter_decompv2’s arguments block. mp_test_solver reports 0 failures. The solver, weighted min-norm, Tikhonov, products, transposes and extraction are correct on every input the tests use.
- **21 defects outside the tested inputs** (Table 2). Ten give a wrong answer with no error or warning. The three to fix first: **B06**, the randomized ID silently caps every off-diagonal rank at 120 (an exact HSS matrix with coupling rank 70 is reproduced with relative error 0.79); **B02/B04**, H(end,end) returns A(1,1) (because size(H) is [1 1]) and H(mask,1:3) with a two-entry mask returns a 64×3 matrix; **B11**, a rank-deficient wide H is “solved” with residual 4.1×10^{-2} and no warning.
- **Five design decisions** should come before new overloads (Section 4): the class name clashes with hm-toolbox’s hss; the property size blocks a size method; the recursive object tree is slow to walk; the compression tolerance needs a definition; and whether to inherit from structuredmat.
- **New overloads** (Section 5): about 40 functions in four priority tiers, plus a list of functions that should *not* be overloaded and why. The six algorithms the plan depends on that are not already in the repository (recompression, block extraction, a cell-array matvec, the adjoint of the stored solve, log-determinant, exact Frobenius norm) were prototyped on the generator representation and checked against dense references: 17 of 17 checks pass (Table 4). The adjoint solve gives tall least squares, $H' \backslash b$ and B/H from the existing ULV factors, with no new factorization.
- **Plan** (Section 7): six phases, each with its files, tests and a check that ends it.

New files (nothing existing was changed), in hss_examples/audit_2026_10/: hss_audit_probes.m tests each defect (one line per bug; a line turns from BUG to ok when the bug is fixed, so it doubles as a regression list; B21 runs in MATLAB only); hss_proto_check.m runs the prototype checks; proto/ holds the prototypes (hssp_*.m).

2 What the class does today

Table 1: Public entry points and the private code behind them, with the `test_hss` tags that cover them.

Entry point	What it does (implementation)	Test tags
<code>hss(A, ...)</code>	Builds from a dense A : halves rows and columns in lockstep to a uniform depth; randomized nested ID per off-diagonal block row/column (<code>inter_decompv3</code> , sparse-sign sketch, threshold or fixed rank). Options <code>blocksize</code> , <code>tol</code> , <code>k</code> , <code>decomp</code> , <code>cutrule</code> , <code>sizeA</code> , <code>powerits</code> , ...	construction, compression
<code>H*X</code>	HSS times dense (<code>hss_matvec</code> : up/down sweep, coefficients in a dictionary)	matvec, rectangular
<code>H1*H2</code>	HSS times HSS on the same tree; ranks add, no truncation (<code>hss_matmat</code>)	matmat
<code>H\B</code>	Square or wide: ULV min-norm solve (<code>hss_ulvminnormsolve</code>); factors cached in <code>H.factorcache</code> ; tall: error	solve, rectangular, minnorm
<code>minnorm</code> , <code>tikhonov</code>	Weighted min-norm and Tikhonov via generator edits (<code>hss_to_gen</code> , <code>hss_gen_weight</code> , <code>hss_gen_augment</code> , <code>hss_from_gen</code>); own cache entries	minnorm
<code>H.', H'</code>	Transpose / conjugate transpose by swapping generators	transpose
<code>H(I,J)</code>	Entry extraction (<code>subsref</code> $\rightarrow$ <code>extract</code>); <code>':'</code> supported	extract
<code>full</code> , <code>spy</code>	Dense matrix (<code>blockbuilder</code>); rank picture	full, construction
<code>clearfactors</code>	Frees the cached factors	—

Not overloaded today: `size`, `end`, `plus`, `minus`, `uminus`, scalar `mtimes`, `dense*H`, `mrdivide`, `inv`, `det`, `norm`, `disp`, `subsasgn`, and every function in Section 5.

3 Audit findings

Severity: **S1** wrong result without error or warning; **S2** error or unusable result on valid input; **S3** performance or portability; **S4** message or clean-up. Numbers come from Octave runs of `hss_audit_probes.m` or of the experiments behind Section 6 (MATLAB timings will differ).

Table 2: Defects, most severe first within each group. “File:line” points at the cause.

ID	Sev	Symptom (measured)	Cause and proposed fix
<i>Construction</i>			

continued

ID	Sev	Symptom	Cause and fix
B06	S1	Exact HSS, coupling rank 70 (leaf block-row rank 140): rank 120 found, matvec rel. error 0.79; rank 100: 1.5; rank 150: 2.8. Coupling ranks 20 and 50 (block-row ranks 40, 100) are exact.	<code>inter_decompv3.m:84,94</code> : threshold mode sketches with $\ell = \min(50, m, n) + 10 = 60$ rows, so the sketch can never see rank > 60 ; ranks in $(60, 120]$ are found only when the one-vector verification (l. 380–406) triggers a redraw, at most twice ($\ell = 60, 120$). Fix: adaptive range finder: if the detected rank reaches $\ell - p$ (the sketch is saturated), double ℓ and resketch until it is not, or until $\ell \geq \min(m, n)$; then the verification is a safety net, not the rank mechanism. Raise <code>hss:notLowRank</code> when $k = \min(m, n)$.
B08	S1	<code>hss(randn(64), cutrule=@(k) k-1)</code> : <code>full(H)</code> is 65×65 .	<code>hss_constructor.m:162,241</code> trusts <code>cutrule</code> ; the depth (l. 75–87) assumes halving, so uneven cuts reach blocks of size 1 and then 0. Fix: require $1 \leq \text{cutrule}(k) \leq k - 1$, and compute the depth by simulating <code>cutrule</code> itself, not halving.
B09	S1	<code>hss(randn(64), sizeA=[32 32])</code> is accepted; <code>H.size</code> is $[32 32]$.	<code>sizeA</code> (l. 13) is an internal value exposed as an option. Fix: remove it from the options (or error when it differs from <code>size(A)</code>).
B13	S1	A NaN entry: built without error; NaN spreads to most entries of every product (112 of 128 in <code>H1</code>).	No input check. Fix: <code>mustBeFinite</code> on <code>A</code> .
B14	S2	<code>hss(sparse A)</code> errors inside <code>inter_decompv3</code> (Octave). In MATLAB the sketch <code>SA</code> is sparse, and sparse <code>qr</code> orders columns to reduce fill, not to reveal rank, so ranks would be wrong (unverified).	Fix: <code>A = full(A)</code> block by block in the constructor (or accept sparse only through the planned matvec-based constructor).
B15	S2	A matrix with fewer than $2 \times \text{blocksize}$ rows or columns (one leaf) has <code>Ir = Ic = []</code> and empty <code>blocksize</code> ; <code>spy(H)</code> errors.	<code>hss_constructor.m:95-97</code> . Fix: set <code>Ir=[1 m]</code> , <code>Ic=[1 n]</code> , <code>blocksize</code> , as <code>hss_from_gen</code> already does.
B20	S3	<code>+hssutil</code> has <code>sparsesign.c</code> and the compiled <code>sparsesign.mexmaca64</code> only: <code>hss(A)</code> fails on Windows, Linux and Intel Macs unless someone runs <code>mex</code> .	Fix: add a plain <code>+hssutil/sparsesign.m</code> (<code>octave_compat/sparsesign_octave.m</code> can serve) and use the MEX when <code>exist(..., 'file')==3</code> .
<i>Indexing and size</i>			
B02	S1	<code>H(end,end)</code> returns <code>A(1,1)</code> ; any <code>end</code> in an index is 1.	No <code>end</code> method; MATLAB's default uses the object-array size, $[1 1]$. Fix: <code>end.m</code> returning <code>H.size(k)</code> (needs B01).
B03	S1	<code>H(n+1,1)</code> , <code>H(0,1)</code> , <code>H(2.5,1)</code> return 0.	<code>extract.m:33-38</code> : indices outside both halves match no <code>ismember</code> mask and keep the zero initialization. Fix: validate indices in <code>subsref</code> (integer, $1 \leq i \leq m$).

continued

ID	Sev	Symptom	Cause and fix
B05	S1	Internal nodes are not matrices: <code>H.A22(1:3,1:3)</code> returns all zeros; <code>H.A22*v</code> and <code>H.A22\ v</code> error; <code>minnorm(H.A22,b,'Weight',w)</code> calls the tree non-uniform.	Nodes keep global <code>Ir/Ic</code> , the global <code>levelcount</code> , and <code>isroot=false</code> . Fix: a <code>subtree(H,path)</code> method that re-roots a copy (shift indices, set depth, add a cache); make the tree properties <code>SetAccess=protected</code> and document <code>subtree</code> as the way to get a block (the probe switches to <code>subtree(H,2)</code> once it exists).
B01	S2	<code>size(H)</code> is <code>[1 1]</code> ; <code>numel</code> , <code>isscalar</code> , <code>length</code> follow.	Property named <code>size</code> , no method. Fix: Decision D2.
B04	S1	<code>H(mask,1:3)</code> with a 64-entry mask holding 2 <code>true</code> s returns a 64×3 matrix (the mask values 0/1 are used as indices); <code>H(k)</code> (linear) errors.	<code>extract.m:33-38</code> . Fix: convert logical masks with <code>find</code> ; map linear indices with <code>ind2sub</code> ; support <code>H(:)</code> only if you want it (it densifies).
B19	S3	<code>H(1,end)</code> builds the whole $n/2 \times n/2$ block: time $\times 9.6$ from $n = 2048$ to 8192 (probe). <code>H(I,J)</code> , 5×4 indices: 0.51 s at $n = 8192$ vs 0.049 s for the prototype.	<code>extract.m:45,50</code> call <code>blockbuilder</code> . Fix: <code>hssp_extract</code> (Section 6.2).
<i>Solves</i>			
B10	S4	Square H with a zero row and column: <code>H\b</code> returns x with residual 4.8×10^{-2} after only the generic “matrix singular to machine precision” warning of a triangular solve (<code>hss_ulvminnormsolve.m:271</code>); MATLAB’s own <code>A\b</code> behaves the same way. The user is not told that H is singular or where.	A singular L_τ is not checked (solver header). Fix: after each <code>lq</code> , compare $\min_i (L_\tau)_{ii} $ with $n \epsilon \ H\ $ (estimate $\ H\ $ once by power iteration); warn <code>hss:singular</code> ; same with <code>rcond</code> for the root LU.
B11	S1	Wide H with two equal rows, random b : residual 4.1×10^{-2} , no warning (open item in the status doc).	<code>hss_ulvminnormsolve.m:128</code> truncates the root SVD silently. Fix: warn <code>hss:rankDeficient</code> when the numerical rank $r < m$, and report r .
B12	S1	<code>minnorm</code> with a singular leaf block in a cell <code>Weight</code> runs without error.	<code>hss_weightops.m:18-24</code> checks block sizes only. Fix: <code>rcond</code> check per block; reject NaN/Inf vector weights.
B07	S2	<code>(H*H*H)\b</code> : leaf rank 18 > 16 leaf rows $\rightarrow$ <code>improperRanks</code> .	<code>hss_matmat</code> adds ranks with no truncation. Fix: <code>compress</code> (Section 6.1); call it inside <code>mtimes/plus</code> when a rank exceeds its block, or always with a tolerance option.
B16	S2	<code>H\3</code> for a one-row H errors (“scalar mat multiplication”).	<code>mldivide.m:12</code> treats any scalar <code>b</code> as scalar division. Fix: test <code>isscalar(b) && H.size(1) ~= 1</code> .
<i>Operators and messages</i>			
B17	S2	(a) <code>2*H</code> , <code>H*2</code> , <code>-H</code> , <code>H+H</code> , <code>x'*H</code> and (b) <code>B/H</code> all error.	Not implemented. <code>x'*H</code> only needs <code>(H.'*x.').'</code> , since <code>transpose</code> exists. See Section 5.

continued

ID	Sev	Symptom	Cause and fix
B18	S4	<code>H*v</code> with a wrong length: “nonconformant arguments (op1 is 16x16, op2 is 21x1)”.	No check in <code>mtimes</code> . Fix: check <code>size(X,1)==H.size(2)</code> , error <code>hss:mtimes:dimension</code> .
B21	S2	Two other <code>@hss</code> folders exist: <code>hm-toolbox's</code> (installed by <code>setup.m</code>) and <code>Solver_files/modified_HMtoolbox_files2019/@hss</code> . Adding <code>structmats</code> “with subfolders” puts both on the path, and <code>hss(A)</code> calls whichever comes first.	Fix: Decision D1.

Lower-priority notes.

- N1. `inter_decompv3`'s header documents `ctype='rank'`, but the code tests for `'k'`; with `'rank'` it silently uses rank 50. Validate `ctype`.
- N2. The ID verification uses one probe vector (`nverify = min(1,n)`, l. 307).
- N3. The constructor's `tol` is relative to each block's own largest pivot, so tiny blocks are resolved to the same relative accuracy as large ones (more rank than a global accuracy needs). See D4.
- N4. `warning('Off-diagonal block may not be low-rank.')` has no identifier, so it cannot be silenced selectively; it floods the test logs.
- N5. Debug output `disp('here rows')`, `disp('here cols')` remains (`hss_constructor.m:326,350`; unreachable for trees the constructor builds).
- N6. Unused properties: `Q`, `S`, `minnormQ`, `nullorrange`; `lowrankrows/cols` are meaningless after `mtimes` or `hss_from_gen`.
- N7. Children store the global `levelcount` (the leaf depth), not their own remaining depth (cause of part of B05).
- N8. Tikhonov with $\lambda \gg \|H\|$: relative error of x grows like $\epsilon\lambda/\|H\|$ (6.7×10^{-10} wide, 1.1×10^{-9} tall at $\lambda = 10^8$; $\leq 1.2 \times 10^{-13}$ for $\lambda \leq 10^2$). This is the conditioning of the augmented system, not a bug; document it ($x \approx H^*b/\lambda^2$ there).
- N9. Lockstep row/column splitting: 64×4096 with `blocksize` 16 gives 16×1024 leaves; 300×20000 with `blocksize` 200 gives no tree at all (one dense leaf). Warn when a leaf's aspect ratio is large; the real fix is separate row and column depths (large change).
- N10. `hss_matvec` needs `dictionary` (R2022b+). The cell-array version (`hssp_matvec`) has no such requirement; in Octave it is 20–115 $\times$ faster and linear in n (the Octave `dictionary` stand-in is not; MATLAB numbers are for you to measure with `hss_proto_check`).
- N11. `test_scalar_multiply_throws` and `test_left_multiply_throws` assert current limitations; they must change when the overloads land.

4 Decisions to make before adding overloads

D1. Class name.

Three `@hss` folders can be on the path (B21). Options: (a) rename to `hssmat` (matches

`toeplitzmat`, `hankelmat`); (b) move it into a package (+`structmats/@hss`, called as `structmats.hss`); (c) keep the name and drop `hm-toolbox` from the path. I recommend (a): it is a one-time search-and-replace and removes the clash for every user.

D2. The size property.

A `size` method is needed for `end`, `size(H,1)` and most of MATLAB’s generic code. A property and a method with the same name are at best confusing and, as far as I know, not allowed in one class; check with a one-line `size.m`. Either way, rename the property (e.g. `sz`) in the same pass as D1. Do not overload `numel`: MATLAB’s documentation advises classes with custom indexing to leave it alone (it determines the number of outputs of indexing; `numArgumentsFromSubscript` is the supported hook).

D3. Storage: object tree or flat generators.

Every node is a full `hss` object with 25 properties (plus the hidden cache). `minnorm` and `tikhonov` already convert to generator cell arrays (`hss_to_gen`) on every new factorization, and every prototype here works on those arrays. Options: (a) keep the tree, add the new code on generators, convert on demand (least change); (b) store the generators as the internal representation and build tree views only for `spy` and debugging (fastest: the cell-array `matvec` is linear and much faster in Octave, B19 disappears, no `dictionary`). (b) touches the constructor, solver and products, so plan it as its own phase (Phase 5) after the overloads exist and are tested on (a).

D4. Meaning of the tolerance.

For `compress`, `plus` and products, decide between a relative tolerance per block (the constructor’s current rule, N3) and an absolute one, $\tau \|H\|_2$ (`hm-toolbox`’s choice). The absolute rule bounds $\|H - \tilde{H}\|_2$ by a small, depth-dependent multiple of $\tau \|H\|_2$ and gives smaller ranks; I recommend it, with $\|H\|_2$ from `normest`.

D5. Inherit from `structuredmat`?

The repo’s abstract class supplies `end` and naive fallbacks that call `full`. Inheriting gives API consistency, but a fallback that densifies a $10^6 \times 10^6$ matrix after a warning is worse than an error. Recommendation: inherit only if every inherited fallback is overridden with an HSS method or an explicit error.

5 Proposed overloads

Notation: n the size, r the maximum rank, $L = O(\log n)$ the depth. “Proto” marks algorithms checked in `hss_proto_check.m`. Priority A is needed first, because other functions and generic MATLAB code rely on it.

Table 3: Overloads to add, grouped by priority.

Function	MATLAB meaning for H	HSS algorithm	Cost	Prio
<i>A. MATLAB semantics (indexing, size, display)</i>				
<code>size(H,dim)</code> , <code>length, ndims</code> , <code>isempty</code>	dimensions	read <code>sz</code> (D2)	$O(1)$	A
<code>end</code>	last index in $H(i,j)$	<code>H.sz(k)</code>	$O(1)$	A

continued

Function	Meaning	Algorithm	Cost	Prio
subsref (rewrite)	$H(I, J)$: logical, linear, end , bounds checks	hssp_extract : partial bases restricted to I, J	$O((I + J)rL + I J)$	A, proto
subsasgn	$H(i, j)=v$	error hss:readOnly (an entry change is a rank-one change; offer lowrankupdate)	–	A
disp/display	show the matrix	one summary: size, depth, leaf sizes, min/mean/max rank, memory, cached factors; never the whole tree	$O(n)$	A
isreal		all generators real	$O(nr)$	A
subtree (new) <i>B. Arithmetic</i>	usable sub-block (B05)	re-rooted copy	$O(\text{size})$	A
uminus, uplus	$-H, +H$	negate all D_τ and B	$O(nr)$	B, proto
mtimes	$sH, Hs; XH$ for dense X	scale $D, B; XH = (H^\top X^\top)^\top$	$O(nr);$ $O(nrp)$	B, proto
mrdivide	$H/s; B/H$	scale; $B/H = (H^* \setminus B^*)^*$: adjoint of H 's factors (wide/square H), forward solve with H^* 's factors (tall H ; min-norm, not MATLAB's basic solution)	$O(nr)$	B
plus, minus	$H_1 \pm H_2$ (same tree); $H \pm D$, D diagonal or sparse diagonal; $H \pm s$	concatenate bases, $B = \text{blkdiag}(B_1, B_2)$, then compress ; diagonal: leaf D_τ only; scalar: MATLAB adds s to <i>every</i> entry, a rank-one update (rank +1 everywhere). A mismatched tree is an error.	$O(nr^2)$	B, proto
compress (new)	re-truncate to tolerance, make proper	orthonormalize up, truncate down (Section 6.1)	$O(nr^2)$	B, proto
lowrankupdate (new)	$H + UV^*$	append U, V to the bases at every level, then compress	$O(n(r+p)^2)$	B
mpower	H^p , integer p	repeated squaring with compress ; $p < 0$: error until inv	$O(nr^2 \cdot \log p)$	B
conj, real, imag		conj all generators; $\text{real}(H) = (H + \bar{H})/2$, then compress	$O(nr^2)$	B
<i>C. Solves and scalar quantities</i>				
mldivide, tall H	least squares H^+b	factor H^* (wide), apply the <i>adjoint</i> of its min-norm solve (Section 6.3); cache under a second key	$O(nr^2)$, then $O(nr)$	C, proto

continued

Function	Meaning	Algorithm	Cost	Prio
$H' \setminus b$	$(H^*)^{-1}b$ or $(H^*)^+b$	for square/wide H : adjoint of H 's own cached factors, no new factorization. Needs ctranspose/transpose to hand H 's cache handle to the result with an “adjoint” flag, assigned <i>after</i> the generator swaps (each set method otherwise gives H' an empty cache)	$O(nr)$	C, proto
logdet (new), det	$\log \det H $, $\det H$	sum of $\log (L_\tau)_{ii} $ over levels plus the root LU; phase from $\det P_\tau \det Q_\tau$ (Section 6.4). det overflows for large n ; document logdet as the one to use	$O(nr^2)$	C, proto
norm (H, 'fro')		exact from orthonormal generators	$O(nr^2)$	C, proto
norm (H), normest	$\ H\ _2$	power iteration on H^*H (matvecs)	$O(nrk)$	C
norm (H, 1/Inf)		exact needs all entries ($O(n^2)$): use normest1 with matvecs, say it is an estimate	$O(nrk)$	C
condest	κ_1 estimate	normest1 on H and on H^{-1} (cached solves)	$O(nrk)$	C
inv	H^{-1}	keep the error for now; later, HSS inversion from the ULV factors or construction from solves (Phase 6). With cached factors, $H \setminus B$ does what users need inv for	–	C
chol	SPD factor	later: symmetric ULV (Xia); halves work, keeps symmetry	$O(nr^2)$	C
<i>D. Structure and spectra</i>				
diag , trace		concatenate / sum leaf diagonals ($k = 0$ only)	$O(n)$	D
tril , triu		zero the upper (lower) couplings, tril of each D_τ ; stays HSS	$O(nr)$	D
blkdiag	$\text{diag}(H_1, H_2)$	new root, zero couplings (equal depths)	$O(1)$	D
sum , mean		$H^T \mathbf{1}$ or $H \mathbf{1}$	$O(nr)$	D
eigs , svds	a few eigen/singular values	wrap function handles: Hx ; shift-invert with cached solves	$O(nrk)$	D
issymmetric , ishermitian		randomized test $\ Hx - H^*x\ \leq \tau \ H\ \ x\ $; say so in the help	$O(nr)$	D

Not to overload, and why.

- **numel**: changes the output count of indexing (D2).
- **horzcat**, **vertcat**: $[H_1 \ H_2]$ is not HSS on any tree that respects both inputs (each block row of H_1 couples to all of H_2 at full rank in general). Only **blkdiag** keeps the structure.
- **eig**, **svd**, **lu**, **qr** in full: $O(n^3)$ or not structure-preserving; **eigs**, **svds** and the ULV factorization cover the uses.
- **expm**, **sqrtn**, **logm**, **funm**: not cheaply HSS-preserving. A later option is a rational approximation through shifted solves, each $O(nr^2)$.
- Elementwise functions (**abs**, **exp**, **.^**): destroy low rank. Hadamard product $H1.*H2$ is HSS with rank $r_1 r_2$; add only if a use appears.
- **double**, **single**, **sparse**: ambiguous for a compressed matrix; keep **full** as the one explicit conversion.

6 Algorithms behind the new functions

Generators as in **hss_to_gen**: leaf blocks D_τ ; leaf bases U_τ, V_τ ; translations R_τ, W_τ with $\hat{U}_\tau = \text{diag}(\hat{U}_\alpha, \hat{U}_\beta) R_\tau$ for children α, β (same for $\hat{V}$); couplings $H(I_\alpha, J_\beta) = \hat{U}_\alpha B_{\alpha\beta} \hat{V}_\beta^*$.

6.1 Recompression (**hssp_orth**, **hssp_compress**)

1. *Orthonormalize, bottom up.* At a leaf, $U_\tau = Q_\tau R_\tau$ (economic QR); replace U_τ by Q_τ and push R_τ into everything that multiplies $\hat{U}_\tau$: the coupling where τ is the row cluster ($B \leftarrow R_\tau B$) and τ 's block of the parent translation. At a parent, QR the updated translation and continue. Same for V with $B \leftarrow B R_\tau^*$. Exact; ranks become $\leq$ block sizes, so this alone fixes **improperRanks**.
2. *Truncate, top down.* With orthonormal bases, the off-diagonal block row of τ is $\hat{U}_\tau S_\tau$ (orthonormal), where $S_\tau = [B_{\tau, \text{sib}}, R_\tau^{(\text{parent})} C_p]$ and $C_p = W_p^* S_p$ is the parent's truncated coefficient. Take the SVD $S_\tau = W \Sigma X^*$, keep k' columns of W , and update $\hat{U}_\tau \leftarrow \hat{U}_\tau W$ (leaf basis or translation columns), $B_{\tau, \text{sib}} \leftarrow W^* B$, parent rows $\leftarrow W^*(\cdot)$. Then the same for columns.

Prototype results: H^3 (leaf rank 18 > 16 rows) compresses to rank 5 with relative error 1.8×10^{-13} , and its solve agrees with dense to 2.1×10^{-13} ; $H + H$ for an exact rank-10 HSS goes back to rank 10; time 0.07, 0.15, 0.30, 0.59, 1.36 s for $n = 2^{12}, \dots, 2^{16}$ (Octave), i.e. linear.

6.2 Block extraction (**hssp_extract**)

Sort the requested rows and columns into leaves; take the matching rows of each leaf basis; move up the tree with $\hat{U}_p(I_p, :) = \text{diag}(\hat{U}_\alpha(I_\alpha, :), \hat{U}_\beta(I_\beta, :)) R_p$; at each node fill the two coupling blocks $\hat{U}_\alpha(I_\alpha, :) B_{\alpha\beta} \hat{V}_\beta(J_\beta, :)^*$. Order and repeats are kept; indices are validated (fixes B03). Times for $H(I, J)$ with $|I| = 5$, $|J| = 4$: 0.020/0.131/0.512 s (current) vs 0.008/0.023/0.049 s (prototype) at $n = 1024/4096/8192$.

6.3 Adjoint of the stored solve: tall least squares, $H' \setminus b$, B/H

For a wide or square W of full row rank, the stored solve is a linear map $\mathcal{S}$ with $\mathcal{S} = W^+$ (memo 1). Each level of $\mathcal{S}$ is a linear map assembled from unitary maps (P_τ^* , Q_τ^*), the isometries Ω_τ^* (economic size reduction), triangular solves (L_τ^{-1}), selections, one product with the transformed matrix $\hat{H}$, and the next level. Taking the adjoint of this combination piece by piece gives $\mathcal{S}^* = (W^+)^* = (W^*)^+$, at the same $O(nr)$ cost, with $\hat{H}^*$ applied by the existing **ctranspose**. Therefore:

- tall H (full column rank): factor $W = H^*$ once; $H^+b = \mathcal{S}^*b$ (least squares);
- square or wide H : $(H^*)^+b = \mathcal{S}^*b$ uses H 's own cached factors, no refactorization;
- $B/H = ((H^*)^+B^*)^*$ for wide or square H (adjoint of H 's factors; this is exactly MATLAB's $B/A = (A' \backslash B')'$). For tall H , H^* is wide: use the *forward* min-norm solve with H^* 's factors, which gives BH^+ ; MATLAB would return a basic solution instead, the same deliberate deviation as $H \backslash b$ for wide H .

Prototype: tall 1024×512 least squares agrees with `pinv` to 6.5×10^{-14} (complex, Table 4) and 7.4×10^{-14} (real, separate run); square $H' \backslash b$ from H 's factors to $\leq 8 \times 10^{-13}$. A proof that the adjoint equals $(W^*)^+$ is two lines from memo 1's Theorem (the solve is exactly W^+ , and $(W^+)^* = (W^*)^+$); it belongs in memo 1's open problems or as a short appendix.

6.4 Log-determinant (`hssp_logdet`)

For square leaves there is no size reduction, and one level gives P^*HQ^* block lower triangular after the same permutation of rows and columns (forced rows couple only to their own forced columns, through L_τ). Hence $\log |\det H| = \sum_\tau \log |\det L_\tau| + \log |\det H_{\text{red}}|$, recursively, ending with the root LU; the phase is the product of $\det P_\tau \det Q_\tau / |\cdot|$ (the stored factors, $\hat{H} = P^*HQ^*$), the phases of $\text{diag}(L_\tau)$ and the root pivots. Prototype: absolute error 4.6×10^{-13} to 1.1×10^{-11} against dense LU on five matrices (real and complex random HSS, an exponential kernel with $\log |\det| = -2151$); phase error $\leq 1.4 \times 10^{-12}$. Use: Gaussian-process likelihoods, where $\log \det$ of a kernel matrix is the expensive part.

6.5 Exact Frobenius norm, cell-array matvec

After orthonormalization the blocks D_τ and $\hat{U}B\hat{V}^*$ are disjoint and the bases orthonormal, so $\|H\|_F^2 = \sum \|D_\tau\|_F^2 + \sum \|B\|_F^2$ (error 5×10^{-14}). `hssp_matvec` stores the up and down coefficients in cell arrays indexed by level and node: 0.007 vs 0.18s at $n = 4096$ and 0.10 vs 11.8s at $n = 65536$ in Octave (the second column is the Octave `dictionary` stand-in; measure MATLAB with `hssp_proto_check`).

Table 4: Output of `hss_proto_check.m` (Octave 8.4). Errors are relative unless marked.

Check	Value	Limit
$2H + (-3 + i)H_B$ via <code>hssp_scale</code> , <code>hssp_plus</code>	4.9×10^{-18}	10^{-13}
<code>hssp_orth</code> : change of the matrix	2.6×10^{-17}	10^{-13}
<code>hssp_orth</code> : $\max_{\tau} \ U_{\tau}^* U_{\tau} - I\ $ (leaves)	7.9×10^{-16}	10^{-13}
<code>hssp_compress</code> (H^3): error	1.8×10^{-13}	10^{-11}
<code>hssp_compress</code> (H^3): leaf rank (was 18)	5	8
<code>hssp_compress</code> (H^3)\b vs dense	2.1×10^{-13}	10^{-10}
<code>hssp_extract</code> : $H(I, J)$, any order, repeats	7.5×10^{-19}	10^{-13}
<code>hssp_extract</code> : a full row	7.8×10^{-18}	10^{-13}
<code>hssp_extract</code> : out-of-range index rejected	yes	yes
<code>hssp_matvec</code> vs $H*x$	1.9×10^{-15}	10^{-13}
<code>hssp_fro</code> vs sum of squares	5.3×10^{-14}	10^{-13}
tall least squares (adjoint) vs <code>pinv</code> , complex 1024×512	6.5×10^{-14}	10^{-11}
square H' \b from H 's factors	5.6×10^{-13}	10^{-11}
<code>hssp_logdet</code> : absolute error	4.6×10^{-13}	10^{-9}
<code>hssp_logdet</code> : phase error	0	10^{-12}
<code>hssp_logdet</code> , complex H : absolute error	6.8×10^{-13}	10^{-9}
<code>hssp_logdet</code> , complex H : phase error	1.4×10^{-12}	10^{-10}

7 Implementation plan

Each phase ends when its tests pass in MATLAB and the matching `hss_audit_probes` lines read ok. New tests go into `test_hss.m` under a tag per phase; small cases compare with dense, large cases ($n \geq 2^{15}$) build from generators and never form the dense matrix.

Phase 0: decisions and safety (small).

D1–D5. Rename class/property, and update `hss_audit_probes.m`, `hss_proto_check.m` and the tests in the same commit (they call `hss(...)` and `H.size`). Fix B06 (adaptive ID), B08, B09, B13, B14, B15, B16, B18, B20, N1, N4, N5. Add `mustBeFinite`, warning identifiers. *Done when:* probes B06, B08, B09, B13–B16, B18, B20, B21 read ok; the constructor tests still pass.

Phase 1: MATLAB semantics (small–medium).

`size`, `end`, `length`, `ndims`, `isempty`, `isreal`, `disp`; rewrite `subsref` on `hssp_extract` (logical, linear, bounds); `subsasgn` error; `subtree`; `SetAccess=protected` on tree properties. *Done when:* B01–B05 and B19 read ok (B19: $H(1, \text{end})$ time ratio ≤ 6 from $n = 2048$ to 8192); $H(\text{end}, \text{end})$, $H(\text{mask}, :)$, $H(k)$ match dense.

Phase 2: arithmetic and compression (medium).

Port `hssp_orth`/`hssp_compress` as `compress(H, tol)`; `uminus`, `uplus`, scalar and dense `mtimes`, `mrdivide` by a scalar, `plus/minus` (HSS, diagonal, scalar), `lowrankupdate`, `mpower`, `conj`, `real`, `imag`; `mtimes(H1, H2)` compresses when a rank exceeds its block (B07). *Done when:* B07, B17a read ok; $\|\text{compress}(H) - H\|_2$ is a small multiple of $\tau \|H\|_2$ on all test families (record the multiple in the test); ranks of $H + H$ equal those of H ; compression time linear to $n = 2^{17}$.

Phase 3: solves and scalar quantities (medium).

Port `hssp_ulv_adjoint` into the solver file (`solve_level_adjoint`); tall $H \setminus b$; $H' \setminus b$ from cached factors (adjoint flag on the transposed object's cache); `B/H`; `logdet`, `det`; `norm` (fro, 2, estimates for 1/Inf), `normest`, `condest`; singularity and rank warnings (B10, B11), weight checks (B12).

Done when: B10–B12 and B17b read ok; tall least squares within $10\kappa\epsilon$ of dense QR on the memo-1 families; $\log\det$ within $10^{-10}|\log\det|$.

Phase 4: structure and spectra (small).

`diag`, `trace`, `tril`, `triu`, `blkdiag`, `sum`, `mean`, `eigs`, `svds`, `issymmetric`, `ishermitian`. *Done when:* each matches dense at $n \leq 2048$; `eigs(H,k,'smallestabs')` reuses cached factors.

Phase 5: flat generator storage (large, optional; D3).

Store generators; rewrite `mtimes`, the constructor output and the solver input on them; keep `hss_from_gen/hss_to_gen` for compatibility. *Done when:* the whole suite passes unchanged and `H*x` timing in MATLAB is no worse than today at every n (Octave suggests much better).

Phase 6: later.

`inv` (HSS inversion), `chol` (symmetric ULV), a `matvec`/entry-based constructor (`hss(@(x) A*x, @(x) A'*x, @(I,J) A(I,J), m, n)`, which never forms A), public `hss.fromGenerators`, geometric splitting and rank-adaptive leaf sizes (open items in the status doc).

8 To confirm in MATLAB

1. Run `runtests('test_hss')` (in `hss_examples/tests`), then `hss_audit_probes` and `hss_proto_check` (add `hss_examples/audit_2026_10` to the path). Octave gives 20 BUG lines and B20 ok (its copy has a `.m` fallback); in MATLAB expect 21–22 (B21 depends on your path) and 17/17 prototype checks.
2. B02: `H(end,end)` in MATLAB (expected identical to Octave: the default `end` sees a 1×1 object).
3. B14: what `hss(sparse A)` does in MATLAB.
4. B10: which warning MATLAB's triangular solve gives for the singular example (Octave: the generic `Octave:singular-matrix`).
5. D2: whether MATLAB accepts a `size` method next to the `size` property.
6. Timing of `hssp_matvec` vs `H*x` (printed by `hss_proto_check`); this decides how urgent Phase 5 is.